



UNIVERSIDAD POLITÉCNICA DE CHIAPAS

Ingeniería en Tecnologías de la Información y la
Innovación Digital

Examen: Diseño de sistemas y Base de Datos

Grupo: C **Docente:** Ramsés Camas Najera

Materia: Bases de Datos Avanzadas

Integrantes del equipo:

Cortés Ruiz Ilya - 243710

Brittany Aurora Hernandez Muñoz - 243707

Luis Alberto Pérez Montiel - 243787

SUCHIAPA, CHIAPAS, 05 DE FEBRERO DE 2026

ÍNDICE

Justificación del Diseño – Caso 3-B.....	3
1. ¿Por qué elegimos este caso?.....	3
Complejidad interesante sin ser abrumadora.....	3
Múltiples relaciones muchos-a-muchos naturales.....	3
2.1 Usamos Tablas Catálogo en lugar de valores fijos (CHECK).....	3
2.2 Implementamos Herencia de Tablas (Organizaciones como tabla padre).....	4
2.3 Separamos Salones como Entidad Débil.....	6
2.4 Tablas Puente con Información Adicional.....	7
Tabla puente: sesion_ponente.....	7
Tabla puente: inscripciones.....	7
2.5 Múltiples Pagos por Paquete.....	8
2.6 Asistentes separados de Paquetes.....	9
3. Restricciones Implementadas.....	10
3.1 Restricciones de Integridad Básicas.....	10
3.2 Restricciones Avanzadas con Triggers.....	10
Trigger 1: Validar que el salón pertenece al centro del evento.....	10
Trigger 2: Validar cupo máximo.....	11
4. Índices para Optimización.....	11
5. Manejo de Casos Especiales.....	11
5.1 División entre cero.....	12
5.2 Datos opcionales.....	12
5.3 Eventos sin patrocinadores.....	12
6. ¿Qué sacrificamos? (Trade-offs).....	13
Trade-off 1: Catálogos vs Simplicidad.....	13
Trade-off 2: Herencia vs Tablas Separadas.....	13
Trade-off 3: Triggers vs Validación en Aplicación.....	13
7. Conclusión.....	14

Justificación del Diseño – Caso 3-B

1. ¿Por qué elegimos este caso?

Elegimos el Caso 3-B (Gestión de Eventos Corporativos) por las siguientes razones:

Complejidad interesante sin ser abrumadora

El caso tiene múltiples actores (empresas, patrocinadores, ponentes, asistentes) que se relacionan de formas diferentes. Esto nos permite modelar de manera más adecuada las distintas entidades existentes en el mundo real, hacía un formato que centralice la información necesaria para la gestión de eventos.

Múltiples relaciones muchos-a-muchos naturales

- Un ponente puede estar en varias sesiones (Con Restricción, no puede estar al mismo tiempo)
- Un asistente puede inscribirse a varias sesiones (Pero no estar simultáneamente en ambas)
- Un patrocinador puede apoyar varios eventos
- Una empresa puede comprar paquetes en distintos eventos

Estas relaciones N-N surgen del modelo relacional y no son forzadas.

2. Decisiones de Diseño Principales

2.1 Usamos Tablas Catálogo en lugar de valores fijos (CHECK)

¿Qué hicimos? En lugar de escribir algo como:

```
tipo_sesion VARCHAR(20) CHECK (tipo_sesion IN ('charla', 'panel', 'taller'))
```

Creamos tablas separadas:

```
CREATE TABLE cat_tipos_sesion (  
  id_tipo_sesion SERIAL PRIMARY KEY,  
  nombre VARCHAR(50) UNIQUE  
);
```

¿Por qué? Si en un futuro se debe agregar un nuevo tipo de sesión, con el uso de check tendríamos que, detener el sistema, actualizar la estructura de la tabla con un ALTER TABLE, y posteriormente reiniciar el sistema completo.

Con la tabla catálogo simplemente hacemos:

```
INSERT INTO cat_tipos_sesion (nombre) VALUES ('Hackathon');
```

Consiguiendo así que no se deban realizar maniobras bruscas dentro de la estructura del sistema.

Ventaja: Podemos guardar más información sobre cada categoría:

- Monto mínimo sugerido para cada nivel de patrocinio
- Descripción y beneficios de cada tipo de paquete
- Características de cada tipo de sesión

2.2 Implementamos Herencia de Tablas (Organizaciones como tabla padre)

Problema encontrado: En el caso original, una empresa podía ser:

- **Cliente:** compra paquetes para enviar empleados
- **Patrocinador:** aporta dinero para apoyar el evento
- **Ambos:** compra paquetes Y patrocina

Si usáramos tablas separadas sin herencia:

-- Tabla 1

```
CREATE TABLE empresas_cliente (  
  razon_social VARCHAR(300),  
  rfc VARCHAR(13),  
  email VARCHAR(150),  
  telefono VARCHAR(20)  
  ...  
);
```

-- Tabla 2 (duplicando los mismos datos)

```
CREATE TABLE patrocinadores (  
  razon_social VARCHAR(300),  
  rfc VARCHAR(13),  
  email VARCHAR(150),  
  telefono VARCHAR(20)
```

...

);

Teníamos problemas estructurales en cuanto al procesamiento de datos, los problemas a destacar son:

1. **Redundancia:** Si una empresa es ambas cosas (Patrocinador y cliente), su información está en dos lugares, puesto que se modifica en la tabla empresas_cliente y patrocinadores.
2. **Inconsistencia:** Si cambian un dato dentro de las tablas (información personal u otros), el cambio debe reflejarse en las tablas que están siendo referenciadas.
3. **Unicidad forzada:** Al estar todos los datos en una tabla, la información pasaba a estar unificada de manera forzosa, generando así un conflicto en la estructura de las queries necesarias.

Nuestra solución (Herencia):

-- Tabla PADRE

```
CREATE TABLE organizaciones (  
  id_organizacion SERIAL PRIMARY KEY,  
  razon_social VARCHAR(300),  
  rfc VARCHAR(13) UNIQUE,  
  email_contacto VARCHAR(150),  
  telefono VARCHAR(20)  
);
```

-- Tabla HIJA 1

```
CREATE TABLE empresas_cliente (  
  id_empresa INT PRIMARY KEY,  
  tipo_industria VARCHAR(100),  
  FOREIGN KEY (id_empresa) REFERENCES organizaciones(id_organizacion)  
);
```

-- Tabla HIJA 2

```
CREATE TABLE patrocinadores (  
  id_patrocinador INT PRIMARY KEY,  
  sitio_web VARCHAR(200),  
  FOREIGN KEY (id_patrocinador) REFERENCES organizaciones(id_organizacion)  
);
```

Ventajas:

- No duplicamos información.
- Una actualización se refleja en todo el sistema.
- Fácil saber si la información es correcta.

Consulta para saber el rol de una organización:

```
SELECT
    o.razon_social,
    o.rfc,
    CASE
        WHEN ec.id_empresa IS NOT NULL THEN 'Es cliente'
        ELSE ''
    END AS rol_cliente,
    CASE
        WHEN p.id_patrocinador IS NOT NULL THEN 'Es patrocinador'
        ELSE ''
    END AS rol_patrocinador
FROM organizaciones o
LEFT JOIN empresas_cliente ec ON o.id_organizacion = ec.id_empresa
LEFT JOIN patrocinadores p ON o.id_organizacion = p.id_patrocinador;
```

2.3 Separamos Salones como Entidad Débil

Un salón no tiene sentido sin su centro de convenciones:

- "Salón Jade" solo tiene significado como "Salón Jade del Poliforum Chiapas"
- Si cerrara el centro, sus salones dejarían de existir.

Implementación:

```
CREATE TABLE salones (
    id_salon SERIAL PRIMARY KEY,
    id_centro INT NOT NULL,
    nombre VARCHAR(100),
    capacidad INT,
    FOREIGN KEY (id_centro) REFERENCES centros_convenciones(id_centro)
        ON DELETE RESTRICT,
    UNIQUE(id_centro, nombre)
);
```

ON DELETE RESTRICT: No queremos que alguien borre accidentalmente el centro y se pierdan todos los salones. Si quieren eliminar el centro, primero deben reubicar o eliminar los salones.

2.4 Tablas Puente con Información Adicional

Tabla puente: sesion_ponente

Un ponente puede estar en varias sesiones (No simultáneamente), y una sesión puede tener varios ponentes. Pero además necesitamos saber **en qué rol participa**:

```
CREATE TABLE sesion_ponente (  
    id_sesion INT,  
    id_ponente INT,  
    id_rol_ponente INT,  
    orden_aparicion INT,  
    notas TEXT,  
    PRIMARY KEY (id_sesion, id_ponente, id_rol_ponente)  
);
```

Misma persona, dos roles diferentes.

Tabla puente: inscripciones

Un asistente puede inscribirse a varias sesiones, pero necesitamos saber:

```
CREATE TABLE inscripciones (  
    id_asistente INT,  
    id_sesion INT,  
    fecha_inscripcion TIMESTAMP,  
    asistio BOOLEAN,  
    hora_registro_asistencia TIMESTAMP,  
    PRIMARY KEY (id_asistente, id_sesion)  
);
```

2.5 Múltiples Pagos por Paquete

¿Qué problema resuelve? En el caso dice: "Grupo Oriente paga 45,000 MXN en dos exhibiciones: 25,000 el 1 de febrero y 20,000 el 28 de febrero"

Opción (un solo campo):

```
CREATE TABLE paquetes (  
    monto_total DECIMAL(12,2),
```

```
    monto_pagado DECIMAL(12,2),  
    saldo_pendiente DECIMAL(12,2)  
);
```

Problemas:

- No sabemos cuántos pagos se hicieron
- No sabemos las fechas de cada pago
- No sabemos el método de cada pago (transferencia, cheque)
- Si reembolsamos uno, no hay historial

Nuestra solución (tabla separada):

```
CREATE TABLE paquetes (  
    monto_total DECIMAL(12,2)  
);
```

```
CREATE TABLE pagos_paquete (  
    id_pago SERIAL PRIMARY KEY,  
    id_paquete INT,  
    monto DECIMAL(12,2),  
    fecha_pago DATE,  
    metodo_pago VARCHAR(30),  
    id_estado_pago INT,  
    referencia_pago VARCHAR(100)  
);
```

2.6 Asistentes separados de Paquetes

¿Cuál es la diferencia?

- **Paquete:** El contrato
- **Asistente:** La persona que va

Relación:

Empresa compra → Paquete (15 lugares) → 12 Asistentes registrados (3 lugares sin usar)

Estructura:

```
CREATE TABLE paquetes (  
    cantidad_lugares INT  
);
```



```
CREATE TABLE asistentes (  
  id_asistente SERIAL PRIMARY KEY,  
  id_paquete INT,  
  nombre_completo VARCHAR(200),  
  email VARCHAR(150),  
  cargo VARCHAR(100)  
);
```

¿Por qué el uso de estas tablas?

Porque necesitamos:

- Saber **quién** va (nombre, cargo, email)
- Enviarles gafetes personalizados
- Trackear a qué sesiones se inscriben
- Saber si dejaron lugares sin usar

3. Restricciones Implementadas

3.1 Restricciones de Integridad Básicas

UNIQUE - Evitar duplicados:

- RFC de organizaciones (no puede haber dos empresas con el mismo RFC)
- Email de ponentes (cada ponente tiene un email único)
- Nombre de centro de convenciones (no dos centros con el mismo nombre)
- Combinación asistente-sesión en inscripciones (no inscribirse dos veces)

NOT NULL - Datos obligatorios:

- Fechas de eventos
- Nombres de personas y organizaciones
- Montos de pagos y paquetes
- Relaciones entre tablas (todas las foreign keys)

CHECK - Validaciones de negocio:

- Capacidad de salones > 0 (no tiene sentido un salón para 0 personas)
- Fecha fin >= fecha inicio (un evento no puede terminar antes de empezar)
- Hora fin > hora inicio en sesiones
- Montos > 0 (no hay pagos negativos ni paquetes gratis, excepto casos especiales)

- Formato de RFC con regex

3.2 Restricciones Avanzadas con Triggers

Trigger 1: Validar que el salón pertenece al centro del evento

Nada impide que programemos una sesión del "Tech Summit en Poliforum Chiapas" usando un salón del "Centro de Convenciones San Cristóbal".

Solución:

```
CREATE TRIGGER trg_validar_salon_evento
BEFORE INSERT OR UPDATE ON sesiones
FOR EACH ROW EXECUTE FUNCTION validar_salon_centro_evento();
```

Este trigger verifica que el salón que estás usando Sí pertenece al centro donde se hace el evento.

Trigger 2: Validar cupo máximo

El caso dice: "El Taller de Datos tiene cupo para 40. Cuando el asistente #41 intenta inscribirse, no debería poder."

Solución:

```
CREATE TRIGGER trg_validar_cupo
BEFORE INSERT ON inscripciones
FOR EACH ROW EXECUTE FUNCTION validar_cupo_sesion();
```

Este trigger cuenta cuántos inscritos hay y no permite pasar del cupo máximo.

4. Índices para Optimización

Índices que creamos:

```
CREATE INDEX idx_sesiones_evento ON sesiones(id_evento);
CREATE INDEX idx_inscripciones_sesion ON inscripciones(id_sesion);
CREATE INDEX idx_pagos_paquete ON pagos_paquete(id_paquete);
```

¿Por qué estos específicamente?

- Buscar sesiones de un evento → muy común en los reportes
- Buscar inscripciones de una sesión → para verificar cupo

- Buscar pagos de un paquete → para calcular saldo

5. Manejo de Casos Especiales

5.1 División entre cero

Al calcular promedios o porcentajes, podemos dividir entre cero lo cual representa un problema:

$tasa_asistencia = asistentes / inscritos$

Solución:

```
CASE
  WHEN COUNT(inscritos) = 0 THEN 0
  ELSE (asistentes * 100.0 / COUNT(inscritos))
END
```

O usando NULLIF:

$asistentes * 100.0 / NULLIF(COUNT(inscritos), 0)$

5.2 Datos opcionales

Problema: No todos tienen email o teléfono (especialmente ponentes o asistentes).

Solución: Permitimos NULL en esos campos:

```
email VARCHAR(150), -- Puede ser NULL
telefono VARCHAR(20) -- Puede ser NULL
```

Pero en las consultas manejamos el caso:

`COALESCE(email, 'Sin email registrado')`

5.3 Eventos sin patrocinadores

Problema: Al calcular "porcentaje de ingresos por patrocinios", si un evento no tiene patrocinadores, podríamos tener errores.

Solución:

LEFT JOIN patrocinios ON evento = patrocinios.evento

Usar LEFT JOIN en vez de JOIN para incluir eventos sin patrocinadores

COALESCE(SUM(monto_patrocinio), 0)

6. Trade-offs

Trade-off 1: Catálogos vs Simplicidad

Ganamos:

- Flexibilidad para agregar valores
- Metadata adicional
- Validación consistente

Perdimos:

- Más tablas (más complejo visualmente)
- Queries con más JOINS
- Más inserts al inicio

Trade-off 2: Herencia vs Tablas Separadas

Ganamos:

- Sin duplicación de datos
- Integridad del RFC
- Una organización puede tener múltiples roles

Perdimos:

- Queries un poco más complejas (necesitas JOIN)
- Concepto más avanzado (no todos lo entienden al principio)

Trade-off 3: Triggers vs Validación en Aplicación

Triggers:

- La base de datos garantiza la regla SIEMPRE
- No importa qué aplicación se conecte
- Más difícil de debuggear
- Menos flexibilidad

7. Views y restricciones

Views Implementadas

1. vw_resumen_financiero_evento

- Calcula ingresos totales por evento separando paquetes y patrocinios
- Muestra porcentaje que representan los patrocinios del total

2. vw_asistencia_sesion

- Compara inscritos vs asistentes reales por sesión
- Calcula tasa de asistencia y clasifica ocupación respecto al cupo

3. vw_ponentes_activos

- Lista ponentes con sus sesiones totales y eventos distintos
- Muestra rol más frecuente y clasifica nivel de actividad

4. vw_inversion_empresa

- Calcula inversión total de cada empresa cliente
- Muestra costo promedio por asistente y estado de pagos

5. vw_balance_pagos

- Lista paquetes con saldo pendiente
- Clasifica estado de pago y días desde la compra

Restricciones Implementadas

Restricciones Implementadas

UNIQUE (8 restricciones)

1. organizaciones.rfc - Un RFC solo puede pertenecer a una organización
2. ponentes.email - Email único por ponente para notificaciones
3. centros_convenciones.nombre - Evita centros duplicados
4. salones(id_centro, nombre) - Salón único dentro de cada centro
5. cat_tipos_sesion.nombre - Tipos de sesión únicos en el catálogo
6. inscripciones(id_asistente, id_sesion) - No inscribirse dos veces
7. sesion_ponente(id_sesion, id_ponente, id_rol_ponente) - No duplicar rol
8. patrocinios(id_evento, id_patrocinador) - Patrocinio único por evento

CHECK (15 restricciones)

1. salones.capacidad > 0 - Capacidad positiva en salones
2. salones.piso >= 0 - Sin pisos negativos
3. eventos: fecha_fin >= fecha_inicio - Evento no termina antes de empezar
4. sesiones: fecha_hora_fin > fecha_hora_inicio - Sesión con duración
5. sesiones.cupo_maximo > 0 OR NULL - Cupo positivo o ilimitado
6. paquetes.cantidad_lugares > 0 - Lugares positivos
7. paquetes.monto_total > 0 - Precio positivo
8. paquetes.estado IN (...) - Estados válidos: activo, cancelado, completado
9. pagos_paquete.monto > 0 - Pago positivo
10. pagos_paquete.metodo_pago IN (...) - Métodos válidos
11. patrocinios.monto_aporte > 0 - Aporte positivo
12. organizaciones.rfc ~ regex - Formato RFC válido
13. empresas_cliente.numero_empleados > 0 OR NULL - Empleados positivos
14. centros_convenciones.capacidad_total > 0 OR NULL - Capacidad positiva
15. asistentes sin CHECK - Datos opcionales permitidos

NOT NULL (25+ campos críticos)

1. organizaciones: razon_social, rfc, fecha_registro - Info básica obligatoria
2. eventos: id_centro, nombre, fecha_inicio, fecha_fin - Datos esenciales
3. sesiones: id_evento, id_salon, id_tipo_sesion, nombre - Relaciones obligatorias
4. salones: id_centro, nombre, capacidad - Info completa de salón
5. paquetes: id_evento, id_empresa, cantidad_lugares, monto_total - Datos de compra
6. pagos_paquete: id_paquete, monto, fecha_pago - Info de pago completa
7. asistentes: id_paquete, nombre_completo - Identificación básica
8. inscripciones: id_asistente, id_sesion, asistio - Relación y estado
9. ponentes: nombre_completo, email - Contacto obligatorio
10. patrocinios: id_evento, id_patrocinador, monto_aporte - Datos de patrocinio

Integridad Referencial (18 Foreign Keys)

1. salones → centros_convenciones - ON DELETE RESTRICT
2. eventos → centros_convenciones - ON DELETE RESTRICT
3. sesiones → eventos - ON DELETE RESTRICT
4. sesiones → salones - ON DELETE RESTRICT

5. sesiones → cat_tipos_sesion - ON DELETE RESTRICT
6. empresas_cliente → organizaciones - ON DELETE CASCADE (herencia)
7. patrocinadores → organizaciones - ON DELETE CASCADE (herencia)
8. paquetes → eventos - ON DELETE RESTRICT
9. paquetes → empresas_cliente - ON DELETE RESTRICT
10. paquetes → cat_tipos_paquete - ON DELETE RESTRICT
11. pagos_paquete → paquetes - ON DELETE RESTRICT
12. pagos_paquete → cat_estados_pago - ON DELETE RESTRICT
13. asistentes → paquetes - ON DELETE RESTRICT
14. inscripciones → asistentes - ON DELETE RESTRICT
15. inscripciones → sesiones - ON DELETE RESTRICT
16. sesion_ponente → sesiones - ON DELETE RESTRICT
17. sesion_ponente → ponentes - ON DELETE RESTRICT
18. sesion_ponente → cat_rols_ponente - ON DELETE RESTRICT
19. patrocinios → eventos - ON DELETE RESTRICT
20. patrocinios → patrocinadores - ON DELETE RESTRICT
21. patrocinios → cat_niveles_patrocinio - ON DELETE RESTRICT

8. Conclusión

Este diseño está pensado para:

- **Crece:** Nuevos tipos, niveles, roles sin modificar estructura
- **Mantener:** Cambios en un solo lugar (herencia)
- **Consultar:** Índices para queries rápidas
- **Proteger:** Triggers y restricciones garantizan reglas de negocio
- **Auditar:** Historial completo de pagos, inscripciones, asistencias

El diseño equilibra:

- Complejidad necesaria vs simplicidad deseable
- Flexibilidad futura vs facilidad de uso actual
- Protección de datos vs velocidad de desarrollo